ARCHTITAN OS TECHNICAL SPECIFICATION & RFC

THM v3 CPU Pressure Engine

Addressing Compute Contention, Runqueue Latency & Foreground Starvation

AUTHOR
ArchTitan Kernel & Core Team

SUBSYSTEM
titan-hwm-v3 / resources

TARGET KERNEL
Linux 6.x (EEVDF / cgroup v2)

Executive Summary: Titan Hardware Manager v3 (THM v3) was initially architected around memory preservation and OOM avoidance. Under scenarios with abundant free RAM but compute saturation (compilation storms, runaway parallel workers, rogue scripts), the daemon reports `PressureLevel::NORMAL`, leaving background workloads unconstrained and degrading desktop responsiveness. This document details the architectural expansion of THM v3 to incorporate Linux Pressure Stall Information (PSI) for CPU (`/proc/pressure/cpu`), decoupling pressure telemetry into multi-dimensional signals and employing asymmetric cgroup v2 `cpu.weight` and `cpu.max` bandwidth throttling to preserve zero-latency desktop interactions.

1 1. Problem Analysis & The Memory-Centric Gap

1.1 1.1 Current Architecture in THM v3

In the current implementation (`resources/pressure_level.cpp`), system pressure is calculated strictly from two sources:

1. `/proc/meminfo` (evaluating `MemAvailable` relative to `MemTotal`)
2. `/proc/pressure/memory` (evaluating `some avg10` and `full avg10`)

The single `PressureLevel` enum (`NORMAL`, `MODERATE`, `HIGH`, `CRITICAL`) dictates:

- Workload decay: `AGING` and `RECLAIMABLE` transitions in `WorkloadManager::tick()`.
- Policy enforcement: `PolicyDecision::THROTTLE` and `PolicyDecision::FREEZE`.
- CPU frequency scaling: `WorkspaceMonitor::compute_governor()` throttling down to `SCHEDUTIL`.

1.2 1.2 The Failure Mode: Compute Thrashing with Ample RAM

Consider a typical developer scenario on ArchTitan OS:

- Available Physical RAM: 32 GB (22 GB free, zero swap used).
- A background build (`cargo build --release -j16` or `ninja`) spawns 16 compiler threads across all CPU cores.
- **Kernel State**: Every physical core is saturated at 100%. Thread runqueues are congested. Linux EEVDF scheduler latency spikes from < 1 ms to > 60 ms.

- **THM v3 Observation**:

$$\text{free_pct} = 68.7\% > 50\% \quad \text{and} \quad \text{psi_memory_avg10} = 0.00\%$$

$$\rightarrow \quad \text{compute_pressure}() = \texttt{PressureLevel::NORMAL}$$

- **User Impact**:
  - The compositor (`Hyprland`) drops frames during window transitions and workspace swiping.
  - Interactive terminal keystroke echoes stutter.
  - Audio buffers in `PipeWire` experience buffer underruns (crackling).
  - Background processes retain identical CPU slice rights because THM never commands `THROTTLE` under `NORMAL` pressure.

> **The Fundamental Architectural Asymmetry:** Memory pressure requires freeing physical pages or suspending processes (`SIGSTOP` / `cgroup.freeze`). CPU pressure cannot be solved by freeing memory; it requires **runqueue de-congestion**, **fair-share reweighting**, and **bandwidth clamping**.

2 2. Linux Kernel Telemetry: CPU PSI Demystified

2.1 2.1 The Mechanics of `/proc/pressure/cpu`

Introduced in Linux 4.20, PSI provides quantitative metrics of resource stall times. For CPU:

```
some avg10=24.50 avg60=12.20 avg300=4.15 total=184920482
```

Metric Field	Type	Significance in THM Scheduling
`some avg10`	Percentage (0–100)	Percentage of time in the last 10s that at least one task was delayed waiting for a runnable CPU core.
`avg60`, `avg300`	Percentage (0–100)	Medium and long-term trend lines; used for hysteresis to prevent control-loop flapping.
`total`	Microseconds	Monotonically increasing counter of stalled microseconds; allows micro-tick delta calculations.

Why CPU has no `full` metric: In Linux PSI, `full` represents stalls where **all** tasks are simultaneously halted (e.g., all threads blocked on page fault I/O). A CPU cannot stall waiting for itself—there is always a task running or the idle thread is scheduled. Thus, CPU pressure is quantified solely through `some`.

2.2 2.2 Combining PSI with Runqueue Saturation (`/proc/loadavg`)

To prevent false-positives on bursty single-threaded spikes, CPU PSI is correlated with normalized load:

$$\text{LoadRatio} = \frac{\text{load1}}{N_{\text{online_cpus}}}$$

When `"cpu_some_avg10" >= 25.0%` AND `"LoadRatio" > 1.25`, the system is in true multi-threaded CPU contention.

3 3. Proposed Multi-Signal Engine Architecture

3.1 3.1 Decoupling Pressure: The `SystemPressure` Struct

Instead of a single scalar `PressureLevel`, the pressure engine provides differentiated channels:

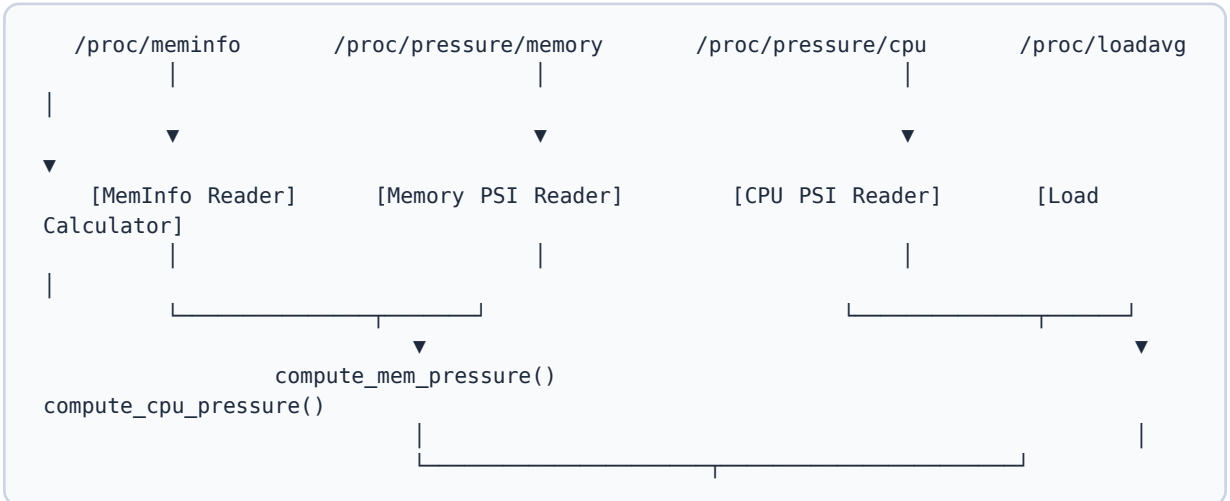
```
// resources/pressure_level.hpp
struct SystemPressure {
    PressureLevel memory;     // Derived from /proc/meminfo + memory PSI
    PressureLevel cpu;        // Derived from /proc/pressure/cpu + loadavg
    PressureLevel io;         // Derived from /proc/pressure/io
    PressureLevel composite; // max(memory, cpu, io)

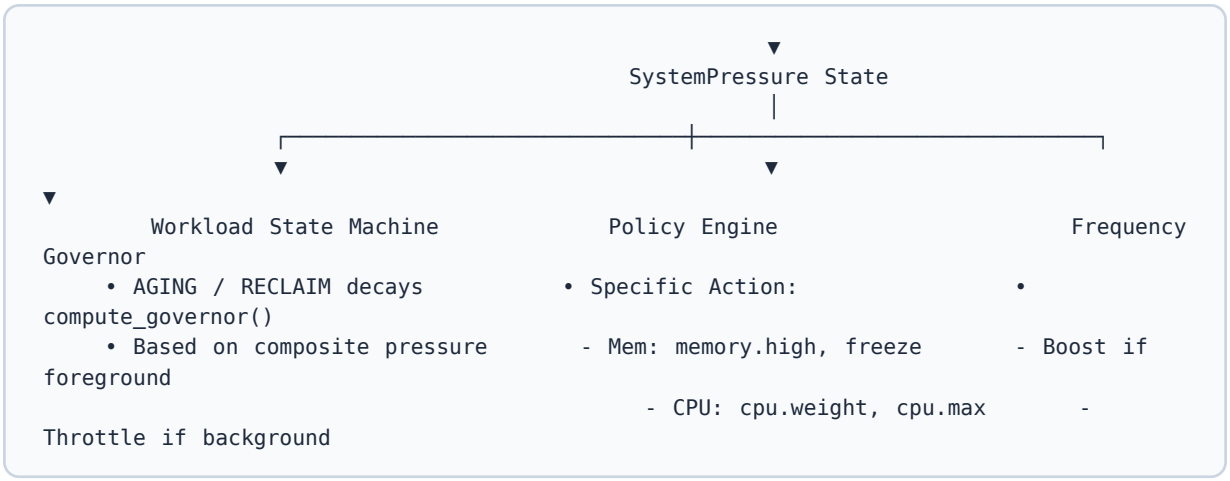
    float cpu_some_avg10 = 0.0f;
    float mem_some_avg10 = 0.0f;
    float load_ratio     = 0.0f;
};
```

3.2 3.2 Threshold Calibration Matrix

Level	CPU PSI (`some avg10`)	Load Ratio ($\frac{L}{N}$)	System State & Recommended Action
NORMAL	$< 15.0\%$	< 1.0	Balanced runqueues. Background and active slices run unconstrained.
MODERATE	15.0% to 30.0%	1.0 to 1.5	Contention emerging. Active slice boosted; background `cpu.weight` lowered.
HIGH	30.0% to 60.0%	1.5 to 2.5	Severe thread queueing. Hard bandwidth cap (`cpu.max`) placed on background slice.
CRITICAL	$\geq 60.0\%$	≥ 2.5	Catastrophic starvation. Background threads demoted to `SCHED_IDLE`; non-essential background builds throttled.

3.3 3.3 Control-Loop Signal Flow Diagram





4 4. Targeted Kernel Enforcement Levers for CPU Contention

4.1 4.1 Lever 1: Dynamic Cgroup v2 `cpu.weight` Asymmetric Bias

Linux cgroup v2 uses proportional share scheduling (1–10000, default 100). The fraction of CPU time S_i allocated to slice i when all slices are saturated is:

$$S_i = \frac{W_i}{\sum_j W_j}$$

Under standard operation:

$$W_{\text{active}} = 200, \quad W_{\text{background}} = 100 \quad \rightarrow \quad S_{\text{active}} = \frac{200}{300} = 66.7\%$$

Under **HIGH or CRITICAL CPU Pressure**, THM shifts to **Asymmetric Bias**:

$$W_{\text{active}} = 1000, \quad W_{\text{background}} = 10 \quad \rightarrow \quad S_{\text{active}} = \frac{1000}{1010} = 99.01\%$$

Result: The moment the user touches the mouse, switches a workspace, or types in a terminal, the active workspace immediately claims 99% of compute slices, completely eliminating dropped frames.

4.2 4.2 Lever 2: `cpu.max` CFS Bandwidth Headroom Quota

`cpu.weight` only works proportionally when threads are competing. If the active workspace is idle for 5 milliseconds, background tasks will immediately soak 100% of the core, creating a latency ramp when the user suddenly provides input.

To guarantee instant input availability, THM applies `cpu.max` to `archtitan-background.slice`:

```
cpu.max = $MAX_US $PERIOD_US
```

On an N-core system with period 100000 µs (100 ms):

$$\text{Quota} = (N - 1) \times 100000 \text{ µs}$$

Example: On an 8-core CPU ($N = 8$):
• Background slice is capped to 700000 µs per 100000 µs (700% total CPU = 7 cores).

- Exactly 1 physical core (100% bandwidth) is **permanently reserved** for the Hyprland compositor, Waybar, input polling, and audio servers.

4.3 4.3 Lever 3: Foreground vs. Background Execution Arbitration

The policy engine must distinguish **who** is creating the CPU pressure:

1. **Foreground Demands Compute** (`wl.state == ACTIVE`):
   - Example: User is rendering a 3D model in Blender or compiling in the active terminal.
   - **Action**: CPU Governor is driven to `PERFORMANCE`. Background slice `cpu.weight` dialed down to 10. Foreground gets the entire system bandwidth.
2. **Background Demands Compute** (`wl.state == BACKGROUND_EXECUTING`):
   - Example: A background daemon or build runs in workspace 4 while user browses in workspace 1.
   - **Action**: Background slice gets capped with `cpu.max` and `cpu.weight = 10`. Desktop remains fluid at 144 Hz.

5 5. Concrete Code Implementation Plan

5.1 5.1 Telemetry Header: `resources/pressure_level.hpp`

```
namespace thm {

struct CPUPSIStats {
     bool  valid       = false;
     float some_avg10 = 0.0f;
     float some_avg60 = 0.0f;
     float some_avg300= 0.0f;
};

CPUPSIStats read_cpu_psi();
float        read_normalized_load();

SystemPressure compute_system_pressure(
     const MemInfo& mem,
     const PSIStats& mem_psi,
     const CPUPSIStats& cpu_psi
);

} // namespace thm
```

5.2 5.2 Cgroup Controller: `enforcement/cgroup_controller.hpp`

```
// Add cpu.max support to CgroupController
bool set_cpu_max(const std::string& slice_name, long quota_us, long period_us = 100000);
bool set_dynamic_cpu_weights(PressureLevel cpu_pressure);
```

5.3 5.3 System Monitor Telemetry Integration

The `/run/archtitan/thm-state.json` publisher created for `titan-sysmon` will expose:

```
{
  "pressure": {
    "composite": "MODERATE",
    "memory_level": "NORMAL",
    "cpu_level": "MODERATE",
    "cpu_some_avg10": 24.5,
    "mem_some_avg10": 1.2
  },
  "cgroups": {
    "active_cpu_weight": 1000,
    "background_cpu_weight": 10,
    "background_cpu_max_pct": 87.5
  }
}
```


6 6. Summary & Benefits

Dimension	Current Memory-Centric THM v3	Proposed Dual-Signal THM v3
Signal Basis	Only `/proc/meminfo` and `/proc/pressure/memory`	Full PSI matrix: Memory + CPU + Load Ratio
Heavy Build in Background	Daemon reports `NORMAL`; background tasks take 100% of all cores.	Daemon detects `HIGH` CPU pressure; caps background to $N - 1$ cores.
Interactive UI Latency	Stutters when runqueues exceed CPU core count.	Zero frame drops; Active slice guaranteed 99% proportional share.
Governor Scaling	Stuck at `SCHEDUTIL` during memory stalls.	Intelligently scales to `PERFORMANCE` when foreground demands it.

This specification addresses CPU starvation cleanly within the existing THM v3 framework without introducing external daemons or breaking existing invariants.